Project 1
Bank Account Management System

Student	Theo Emmanuel Bernard Bacconnet
Erasmus University	University of Split - FESB
Home University	Clermont Auvergne INP - ISIMA
Course	Programming in Java (FELP11)
Academic Year	2025/2026
Submission Date	09/06/2026

Table of contents

1. Introduction

This report documents the design and implementation of a Java desktop application that simulates a simplified bank account management system. The project was developed during my Erasmus exchange semester at the University of Split - FESB for the Programming in Java course.

The idea of this project is to simulate a real banking interface that a bank employee could use. It supports three account types, financial transactions, transaction history, and data persistence between sessions.

Main Features

- Create customers and open CheckingAccount, SavingsAccount or businessAccount
- Deposit, withdraw, and transfer funds between accounts
- view full transaction history per account, sortable by date or amount
- Search customers by name or email, filter accounts by type and balance range
- Sort customers alphabetically and accounts by balance
- Close accounts (marked inactive, no further transaction allowed)
- Apply monthly rules: interest accrual for savings accounts, withdrawal counter reset
- Automatic CSV persistence, data is saved and reloaded on every startup

Design Choices

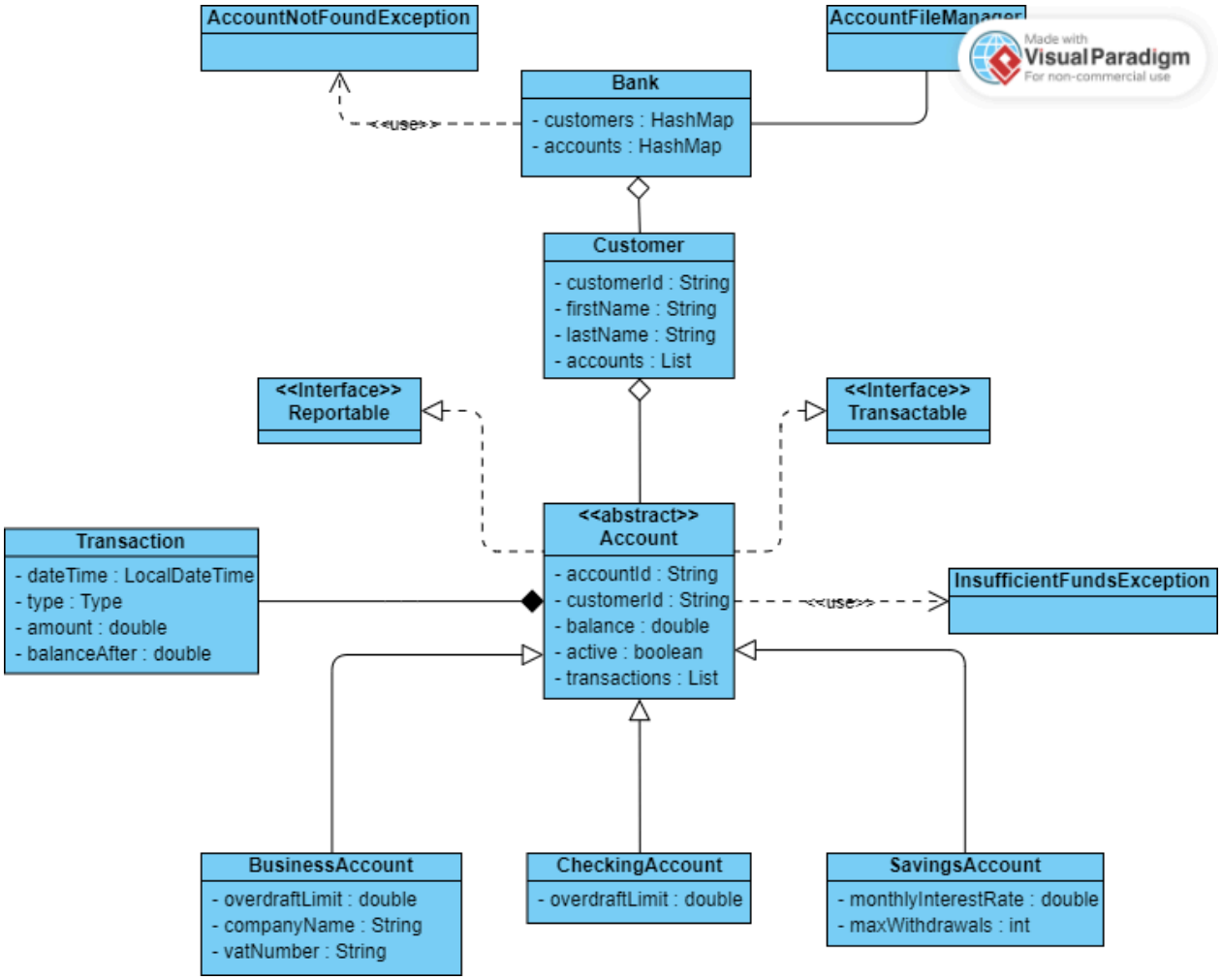
The main design decision was to keep the GUI completely decoupled from the business logic. The BankGUI class only calls methods on Bank and never accesses the internal data structures directly. This makes the code easier to test and maintain.

The abstract Account class centralises all shared logic (deposit, transaction recording, active check) so that subclasses only need to implement their specific withdrawal rules and monthly behaviour. This avoids code duplication across the three account types.

For persistence, three separate CSV files are used (customers, accounts, transactions) instead of a single file. This makes it easier to load data in the correct order and to debug issues.

2. UML Class Diagram

The diagram shows all classes, interfaces, inheritance relationships, and key associations.



3. Implementation Details

Account Hierarchy

The abstract class Account centralises all shared logic: deposit validation, transaction recording, and the active flag. Withdrawal logic is left to each subclass because the rules differ (CheckingAccount checks an overdraft limit, SavingsAccount checks both the balance and a monthly withdrawal counter, and BusinessAccount checks a higher overdraft limit). The abstract method applyMonthlyRules() is called using polymorphism on all active accounts when the user triggers it from the File menu.

Bank, Central Coordinator

Bank owns an ArrayList<Customer> to store all customers, and a HashMap<String, Account> mapping account IDs to Account objects for direct O(1) operations. Customer searches iterate through the list using findCustomer(), while account searches use findAccount() with the HashMap. All business operations (deposit, withdraw, transfer, close) go through bank, which then delegate to the relevant Account object. The GUI never accesses the internal data structures directly.

AccountFileManager, Persistence

Three CSV files are maintained in the data/ folder. On start, customers are loaded first, then accounts (which are linked to their customer), then transactions (which are linked to their account). This order is important because each step depends on the previous one. this was not obvious at first and caused some NullPointerExceptions during early testing before I understood the dependency chain. The balance is restored using setBalanceDirectly() to avoid creating a duplicate opening transaction. Indeed, if we restore the balance by using deposit(), then we will create a new transaction and that's not what we want.

BankGUI, Swing Interface

The main window uses a JTabbedPane with four tabs: Dashboard, Customers, Accounts, and Transactions. Each tab uses a JTable with a DefaultTableModel. All user operations open JDialog instances that validate input before calling Bank methods. Errors are always shown in a JOptionPane dialog rather than printed to the console.

4. Java Concepts Applied

OOP - Classes & Objects (NJ 3-4)

The project defines 10 classes, each with private fields accessible only through public getters and setters. Every class has a dedicated constructor. For example, Account, Customer, Transaction, Bank and AccountFileManager all follow encapsulation, no field is ever accessed directly from outside its class.

```
private double balance;

public double getBalance() {
    return balance;
}

public void setBalance(double balance) {
    this.balance = balance;
}
```

Inheritance & Polymorphism (NJ 6)

Account is an abstract class extended by CheckingAccount, SavingsAccount and BusinessAccount. Each subclass override withdraw() with its own validation rules. The method applyMonthlyRules() is declared abstract in Account and called polymorphically on every active account in Bank, without needing to know the concrete type.

```
for (Account account : accounts.values()) {
    if (account.isActive()) {
        account.applyMonthlyRules();
    }
}
```

Interfaces (NJ 7)

I defined two interfaces. Transactable declares deposit(), withdraw(), and getBalance(). Reportable declares getTransactions() and getSummary(). Both are implemented by the abstract class Account, which means all three account types inherit these contracts automatically.

```
public abstract class Account
        implements Transactable, Reportable {
    // ...
}
```

Exception Handling (NJ 9)

Two custom exceptions are defined: InsufficientFundsException, thrown when a withdrawal exceeds the available balance or overdraft limit, and AccountNotFoundException, thrown when a customer or account search fails. In AccountFileManager, FileNotFoundException and IOException are caught separately in every load method, using try-with-resources to make sure that files are always closed properly.

```
} catch (AccountNotFoundException | InsufficientFundsException ex) {
    JOptionPane.showMessageDialog(dialog, ex.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
} catch (NumberFormatException ex) {
    JOptionPane.showMessageDialog(dialog, "Please enter a valid amount.",
            "Error", JOptionPane.ERROR_MESSAGE);
} catch (IllegalArgumentException ex) {
    JOptionPane.showMessageDialog(dialog, ex.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
} catch (IllegalStateException ex) {
    JOptionPane.showMessageDialog(dialog, ex.getMessage(),
            "Error", JOptionPane.ERROR_MESSAGE);
}
```

Collections (NJ 11)

Bank uses an ArrayList<Customer> to store all customers, and a HashMap<String, Account> mapping account IDs to Account objects for O(1) research. Customer holds its accounts in an ArrayList<Account>. Two sorting methods use Collections.sort() with a custom Comparator: one sorts customers alphabetically by last name, the other sorts accounts by balance in descending order.

```
private List<Customer> customers = new ArrayList<>();

private Map<String, Account> accounts = new HashMap<>();

public List<Customer> getCustomersSortedByName() {
    List<Customer> sorted = new ArrayList<>(customers);
    Collections.sort(sorted, new Comparator<Customer>() {
        @Override
        public int compare(Customer c1, Customer c2) {
            return c1.getLastName()
                    .compareToIgnoreCase(c2.getLastName());
        }
    });
    return sorted;
}
```

File I/O (NJ 10)

AccountFileManager handle all persistence using three CSV files stored in the data/ folder. Data is written with BufferedWriter and read with BufferedReader. The 3 files are loaded in a specific order on startup (customers first, then accounts, then transactions) because each step depends on the previous one.

Code snippet :

```
try (BufferedWriter writer =
        new BufferedWriter(new FileWriter(CUSTOMERS_FILE))) {
    for (Customer customer : customers) {
        writer.write(customer.getCustomerId() + ","
                + customer.getFirstName() + ","
                + customer.getLastName());
        writer.newLine();
    }
} catch (IOException e) {
    System.err.println("Error saving: " + e.getMessage());
}
```

Helper Classes (NJ 11)

LocalDateTime.now() is called each time a transaction is created, giving every entry an accurate timestamp. String.format() with the %.2f EUR pattern is used in the GUI and in getSummary() to display monetary values with two decimal places.

```
transactions.add(new Transaction(
    LocalDateTime.now(), type, amount, balance));

String.format("%.2f EUR", account.getBalance());
```

GUI - Swing (NJ 12)

The main window is a JFrame containing a JtabbedPane with four tabs: Dashboard, Customers, Accounts, and Transactions. Each tab uses a JTable with a DefaultTablemodel. User operations open modal JDialog instances with JTextField, JComboBox, and JButton components. A JMenuBar with File and help menus provides save and monthly rules and exit actions.

```
JTabbedPane tabbedPane = new JTabbedPane();
tabbedPane.addTab("Dashboard",    buildDashboardPanel());
tabbedPane.addTab("Customers",    buildCustomersPanel());
tabbedPane.addTab("Accounts",     buildAccountsPanel());
tabbedPane.addTab("Transactions", buildTransactionsPanel());
setJMenuBar(buildMenuBar());
```

5. Testing

Testing was done manually by running the application and going through all seven core operations listed in the project specification.

Operations Tested

- Create customer : tested with valid data and with empty fields (error dialog shown)
- Create account : tested all three types with valid and invalid inputs (negative balance, wrong overdraft sign, invalid interest rate)
- Deposit : tested positive amounts accepted,negative amounts correctly rejected with error dialog
- Withdraw : tested within balance, at the overdraft limit and beyond it (InsufficientFundsException shown in GUI), negative amounts rejected
- Withdraw on inactive account : IllegalStateException correctly shown in GUI
- Transfer : tested between two different accounts and same account transfer correctly rejected
- Transaction history : verified correct order and balance after values, sorting by date and amount both verified
- Search and filter : name search, type filter, and balance range filter all verified
- Close account : confirmed that further operations on a closed account are blocked

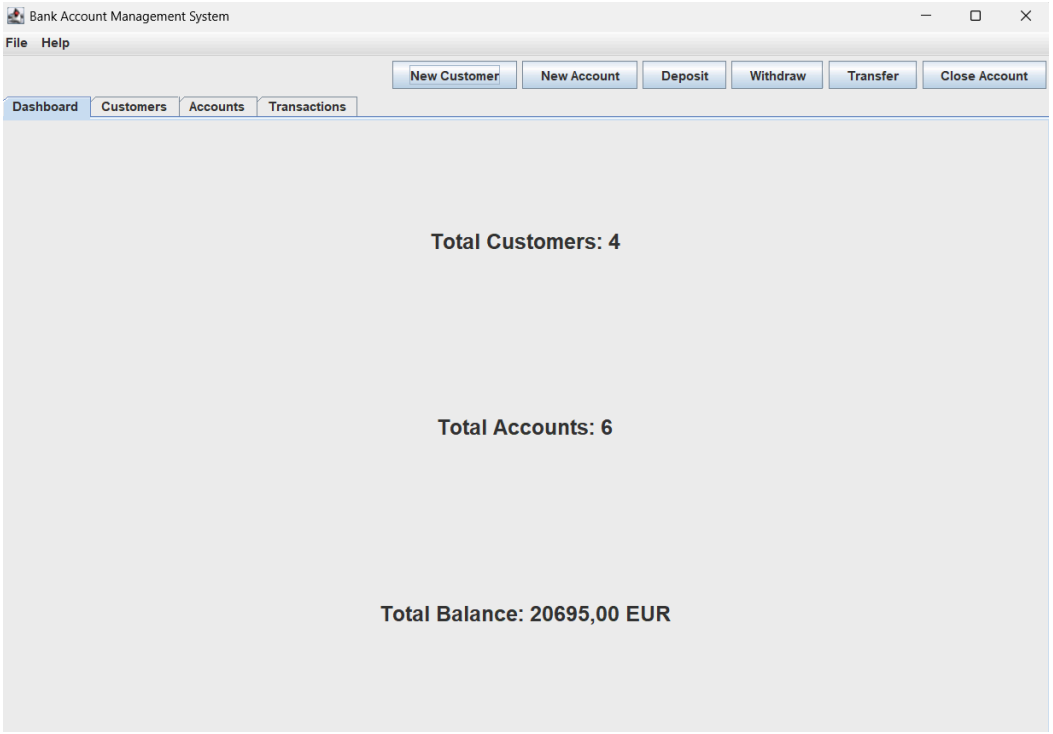
Persistence Test

After creating accounts and performing transactions, the application was closed. On restart, all customers, accounts, balances, and transaction histories were correctly reloaded from the CSV files in the data folder. A new customer was then created to confirm that ids continued from where they left off, with no duplicates.

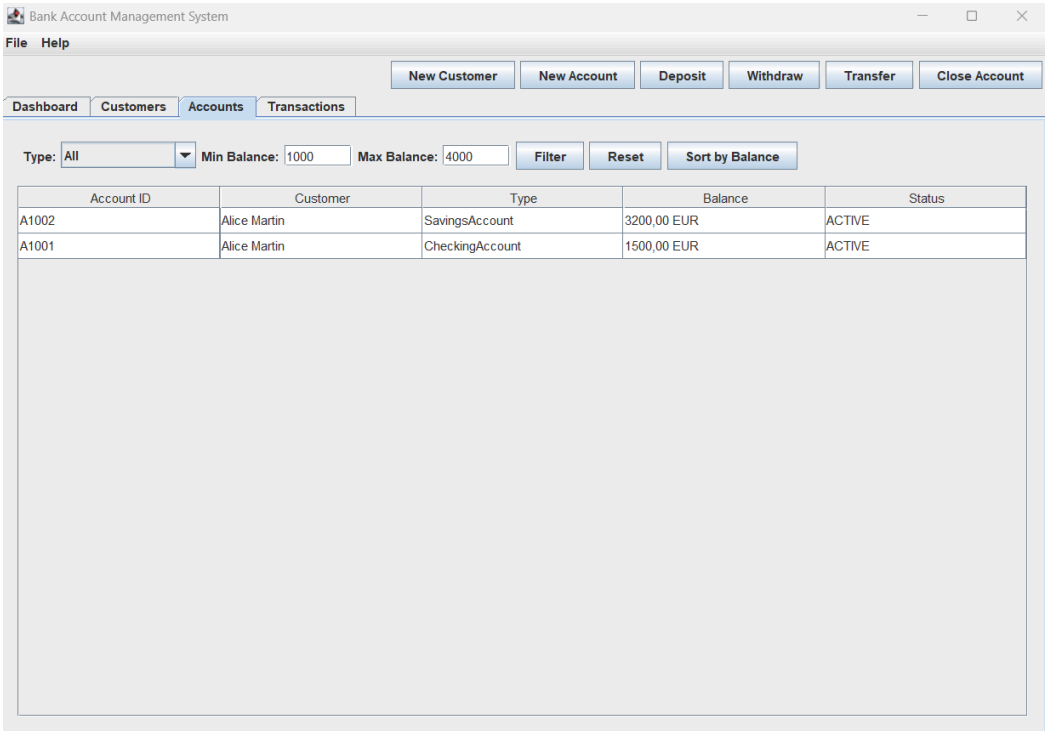
Exception Verification

- InsufficientFundsException : triggered by withdrawing beyond the overdraft limit on CheckingAccount and BusinessAccount, and by exceeding the monthly withdrawal limit on SavingsAccount
- IllegalStateException : triggered by depositing or withdrawing on a closed account
- IllegalArgumentException : triggered by entering a negative deposit amount
- NumberFormatException : triggered by entering non-numeric input in amount or balance fields

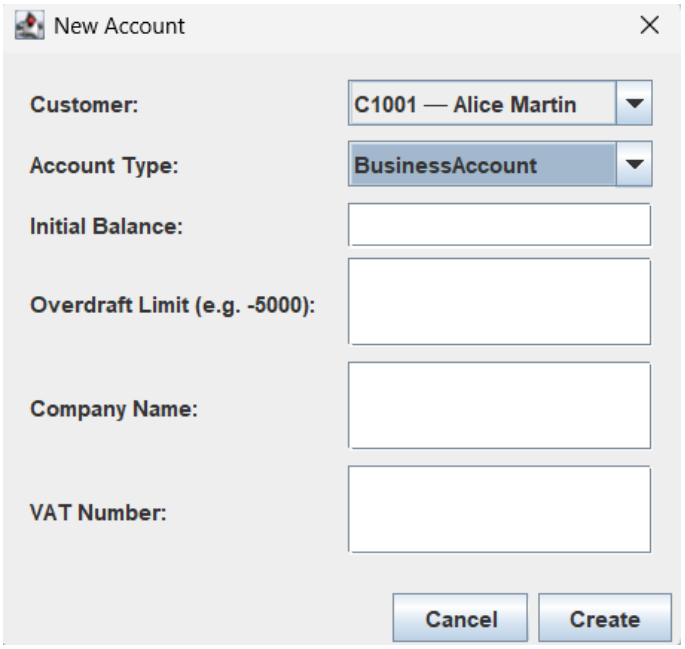
GUI Screenshots



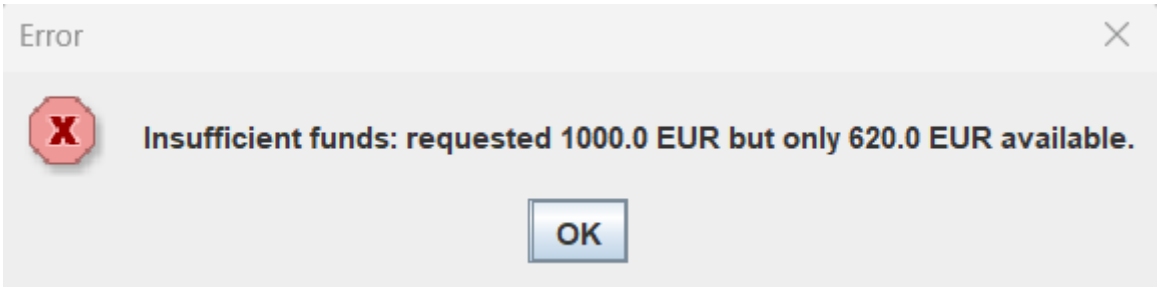
Dashboard showing number of customers, number of accounts and total balance



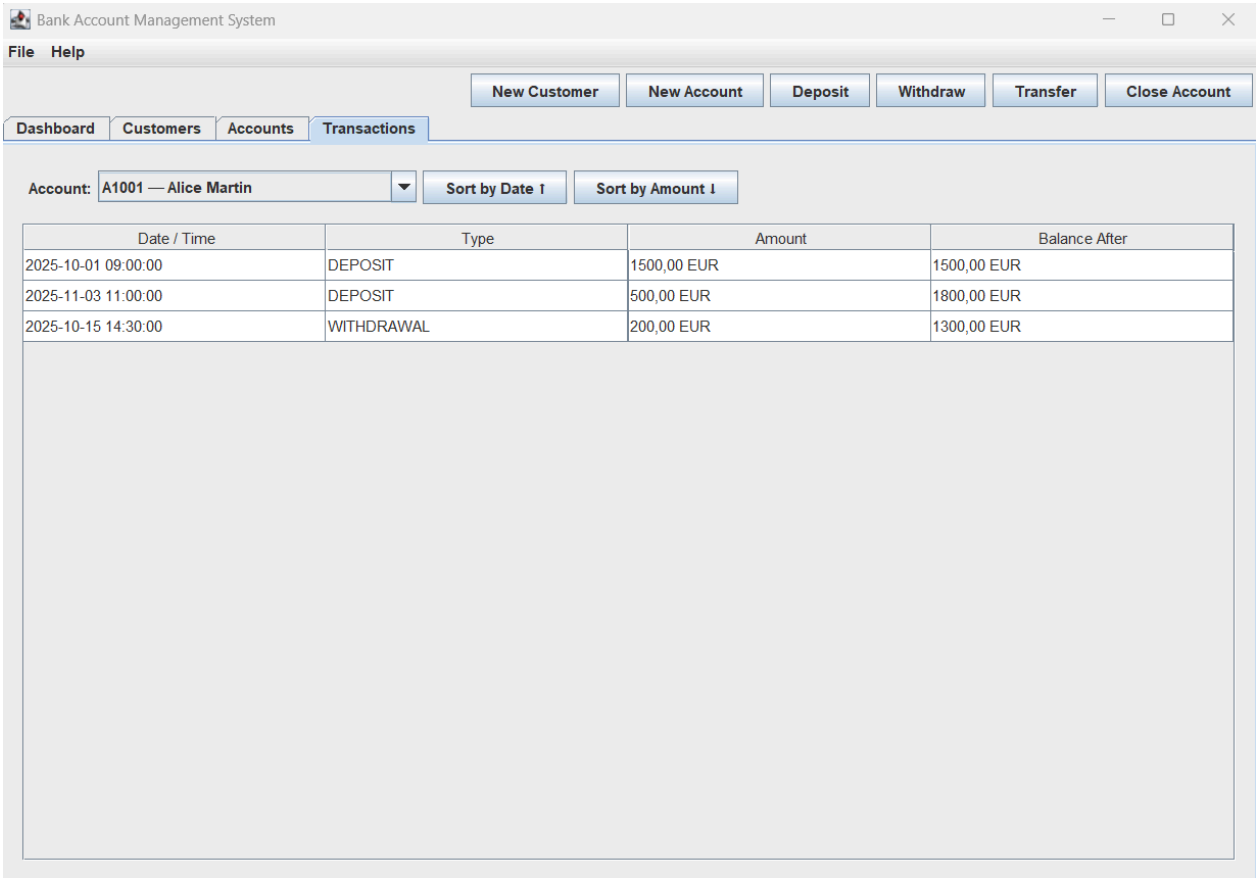
List of Accounts with the balance filter used (accounts with a balance between 1000 and 4000 euros)



Dialog to create a new account, here with the Business Account selected



Error message due to insufficient funds during a withdrawal



List of transactions sorted by amount

6. Challenges and Solutions

Restoring balance on load

At first I tried to simply call deposit() to restore the balance when loading data from the CSV files. I quickly realized this was creating duplicate transactions every time the application restarted. The method deposit() does two things at once, it updates the balance and records a new DEPOSIT transaction, which means the balance would accumulate incorrectly after each restart. After some debugging I decided to add two dedicated methods: setBalanceDirectly(), which set the balance without recording anything, and addTransactionDirectly(), which appends a transaction to the list without modifying the balance. These two methods are only ever called by AccountFileManager during loading.

Avoiding duplicate IDs

I initially set both counters to 1000 without thinking about what would happen after a restart. When I tested persistence for the first time, I noticed that new IDs were colliding with existing ones (the counters were resetting to 1000 on every startup and silently overwriting existing data). I then added a syncCounters() method called at the end of the Bank constructor, after loading. It iterates through all loaded customers and accounts, finds the highest numeric ID and sets the counters so that the next generated ID is always greater than any existing one.

Serialising polymorphic types

The three account subclasses each have different fields: CheckingAccount has an overdraft limit, SavingAccount has an interest rate and a withdrawal counter, and BusinessAccount has an overdraft limit, a company name, and a vat number. The challenge was to save and restore these fields without knowing the concrete type at load time. I decided to store the class name as a field in accounts.csv and implement extraFieldsToCsv() in each subclass to serialise its specific fields. On load, AccountFileManager reads the type string and use a switch to instantiate the correct subclass before restoring the remaining fields.

Dynamic fields in the New Account dialog

This was probably the most challenging part of the GUI. The dialog needed to show different input fields depending on the account type selected: one field for CheckingAccount, two for SavingsAccount, and three for BusinessAccount. I first tried to hide and show components individually, which caused layout issues and left empty spaces in the form. I found that swapping entire panels using removeAll(), revalidate(), and repaint() was much cleaner. The dialog size is also adjusted accordingly when the type changes.

Loading data in the correct order

When the application starts, data must be loaded in a specific sequence: customers first, then accounts, then transactions. This was not obvious at first and caused some NullPointerExceptions during early testing before I understood the dependency chain. If accounts are loaded before customers, loadAccounts() has no customers to attach them to, and if transactions are loaded before accounts, loadTransactions() has no accounts to attach them to. I enforced a strict order in loadAll() and passed the already populated HashMap<String, Account> directly to loadTransactions() allowing each transaction to be matched to its account in O(1) without iterating through the entire list.

7. References

Oracle. (2024). *Trail: Creating a GUI with Swing*. Retrieved from https://docs.oracle.com/javase/tutorial/uiswing/

Oracle. (2024). *Trail: Collections*. Retrieved from https://docs.oracle.com/javase/tutorial/collections/

Stack Overflow. (2024). *Various Java syntax questions*. Retrieved from https://stackoverflow.com

Geeks for Geeks. (2024). *Introduction to Java Swing*. Retrieved from https://www.geeksforgeeks.org/java/introduction-to-java-swing/

Bro Code. (2023). *Java Swing Full Course* [Video]. YouTube. Retrieved from https://www.youtube.com/watch?v=Kmgo00avvEw

Baeldung. (2024). *Exception Handling in Java*. Retrieved from https://www.baeldung.com/java-exceptions

Coding with John. (2022). *Map and Hashmap in Java - Full tutorial* [Video]. YouTube. Retrieved from https://www.youtube.com/watch?v=H62Jfv1DZMc

Oracle. (2024). *Java I/O Tutorial - Reading and Writing Files*. Retrieved from https://docs.oracle.com/javase/tutorial/essential/io/